

Machine Learning Development Life Cycle (MLDLC) - Complete Notes

Introduction to MLDLC

Until now, most topics in Machine Learning (ML) have focused on the **What** and **Why**. This topic marks the first time we shift our focus to **How**—the actual process of doing Machine Learning.

Why is this important?

- It acts as a **Roadmap**. Future videos and learning paths will stem from these 9 steps.
- In Computer Science, we have **SDLC** (Software Development Life Cycle) which guides how software is built from start to finish. Similarly, **MLDLC** provides the guidelines for building ML-based software products.
- **Common Student Mistake:** Most students think that training a model and checking accuracy is the end of the job. In the industry, companies look for people who can build an **End-to-End** product.

1. Framing the Problem

Before writing code or collecting data, you must define the problem. Since you are working for a company/client, you cannot change the goal midway because it involves significant costs.

Key Questions to Answer at this Stage:

1. **Exact Problem:** What are we trying to solve? (e.g., a recommendation system, a bank's loan prediction, or student marks prediction).
2. **Target Customers:** Who will use this?
3. **Feasibility:** How much will it cost? How many people are needed in the team?
4. **Technical Approach:** * Is it Supervised or Unsupervised learning?
 - Will it run in **Batch Mode** (offline) or **Stream Mode** (real-time)?
 - What kind of algorithms might help?
 - Where will the data come from?

Key Point: Getting a mental idea of the entire process here prevents wasted effort later.

2. Data Gathering

Without data, Machine Learning is impossible. In college, we get clean CSV files from Kaggle, but in the industry, data gathering is much harder.

Common Data Sources & Methods:

- **Direct Files:** Sometimes provided as CSV or Excel.

- **APIs:** Writing Python code to hit an API and fetch data in **JSON** format, then converting it to a usable format.
- **Web Scraping:** Using code to extract data from websites (e.g., Trivago scraping prices from different hotel sites).
- **Data Warehouses:** Large companies store data in databases. We use **ETL (Extract, Transform, Load)** processes to move data to a "Data Warehouse" for analysis.
- **Big Data:** Using tools like Spark or Hadoop clusters to fetch massive amounts of data.

3. Data Preprocessing

Raw data is often "dirty" or "unclean" and cannot be fed directly into an ML model.

Common Tasks in Preprocessing:

1. **Remove Duplicates:** Deleting repeated rows.
2. **Handle Missing Values:** Filling in gaps or removing incomplete rows.
3. **Handle Outliers:** Removing data points that are logically impossible or extreme.
4. **Feature Scaling (Standardization/Normalization):** * *Example:* If one column has values in millions (e.g., Salary) and another in decimals (e.g., Experience), the algorithm might get confused. We bring them to the same scale (like 0 to 1).

Goal: Transform the data into a format that the algorithm can easily consume.

4. Exploratory Data Analysis (EDA)

EDA is where you "analyze" the data to find hidden relationships between input and output.

What happens in EDA?

- **Visualizations:** Plotting graphs and charts.
- **Univariate Analysis:** Analyzing one column at a time (checking mean, standard deviation, etc.).
- **Bivariate Analysis:** Comparing two columns to see how they affect each other.
- **Multivariate Analysis:** Comparing 3 or more columns.
- **Handling Imbalanced Data:** If you are classifying "Cat vs Dog" and have 1000 Cat images but only 10 Dog images, you must balance this.

Teacher's Tip (The "Axe" Analogy): If you have 6 hours to cut a tree, spend 4 hours sharpening the axe. EDA is the sharpening process. The more time you spend understanding data, the easier the modeling becomes.

5. Feature Engineering & Selection

Features are simply the **Input Columns** of your data.

Feature Engineering

Creating new columns from existing ones to make the model's job easier.

- *Example:* Instead of having two separate columns for "Number of Rooms" and "Number of Bathrooms," you combine them into a single "Square Feet" column.

Feature Selection

Deciding which columns are actually important.

- **Why select?** 1. Not every input affects the output.
2. More columns = More time taken to train the model.

6. Model Training & Evaluation

Once the data is ready, you start the "Machine Learning" part.

The Process:

- **Training:** We don't just use one algorithm. We try several (e.g., Random Forest, Linear Regression, SVM).
- **Evaluation:** We use **Performance Metrics** to see which model is performing best.
 - *Regression:* Mean Squared Error.
 - *Classification:* Accuracy Score.
 - *Clustering:* Silhouette Index.

7. Model Selection & Hyperparameter Tuning

After trying many models, you pick the best one. But you can still make it better.

The Process:

- **Hyperparameter Tuning:** Adjusting the internal settings of the algorithm (like tuning the "Picture Mode" or "Volume" on a TV) to get the best performance.
- **Ensemble Learning:** Combining multiple algorithms to create one "Super Model" (Techniques like Bagging, Boosting, or Stacking).

8. Deployment

Now that the model is ready, it must be turned into a software product so users can access it via a Website or Mobile App.

The Deployment Logic (Line-by-Line Breakdown)

While the teacher did not write code on a terminal, he explained the architectural flow of deployment:

1. Create a Binary File:

Use a tool like Pickle to save the trained model as a .pkl (binary) file.

- *Explanation:* This converts the Python object into a file that can be loaded later without retraining.

2. Convert to API:

Wrap the model in a Python web framework (like Flask or Django) to create an API.

- *Explanation:* An API allows the model to receive inputs and send back results (usually in JSON format).

3. Server Communication Flow:

- **User Input:** User fills a form on a website.
- **Server:** The Website sends this data to the Python Server.
- **Prediction:** The Python code loads the Binary File, processes the input, and generates a prediction.
- **Response:** The result is sent back to the User's screen.

Platforms used: Heroku, AWS (Amazon), GCP (Google Cloud), or Azure.

9. Testing & Optimization

The job isn't over once the model is online. You must monitor it.

Key Steps:

- **Beta Testing:** Roll out the new feature to a small group of loyal customers first and get feedback.
- **A/B Testing:** Compare the performance of the new model against the old version.
- **Backups:** Keep backups of both the Data and the Model.
- **Retraining Frequency:** Models degrade over time (called **Rotting** or **Concept Drift**).
 - *Example:* If a model detects face masks, and people start wearing new styles of masks that look like faces, the model will fail. You must decide if you need to retrain the model weekly or monthly.

Final Goal: Automate the entire process to ensure it runs perfectly with minimal manual effort.

Summary / Key Points to Remember

- **MLDLC** is the complete process from "Problem Idea" to "Automated Product."
- **EDA** and **Preprocessing** take the most time but are the most important.
- **Deployment** is what turns a "Data Science Project" into a "Software Product."
- **Concept Drift:** Data changes over time, so models must be retrained.